arXiv:2510.04309v1 [cs.LG] 5 Oct 2025

ACTIVATION STEERING WITH A FEEDBACK CONTROLLER

Dung V. Nguyen[1]* **Hieu M. Vu**[3]* **Nhi Y. Pham**[2]* **Lei Zhang**[1]† **Tan M. Nguyen**[1]†
[1] Department of Mathematics, National University of Singapore
[2] VinUniversity [3] Independent
dungnv@u.nus.edu, nhiiyennphamm@gmail.com, vmhieu17@gmail.com,
{matzhlei,tanmn}@nus.edu.sg

ABSTRACT

Controlling the behaviors of large language models (LLM) is fundamental to their safety alignment and reliable deployment. However, existing steering methods are primarily driven by empirical insights and lack theoretical performance guarantees. In this work, we develop a control-theoretic foundation for activation steering by showing that popular steering methods correspond to the proportional (P) controllers, with the steering vector serving as the feedback signal. Building on this finding, we propose Proportional-Integral-Derivative (PID) Steering, a principled framework that leverages the full PID controller for activation steering in LLMs. The proportional (P) term aligns activations with target semantic directions, the integral (I) term accumulates errors to enforce persistent corrections across layers, and the derivative (D) term mitigates overshoot by counteracting rapid activation changes. This closed-loop design yields interpretable error dynamics and connects activation steering to classical stability guarantees in control theory. Moreover, PID Steering is lightweight, modular, and readily integrates with state-of-the-art steering methods. Extensive experiments across multiple LLM families and benchmarks demonstrate that PID Steering consistently outperforms existing approaches, achieving more robust and reliable behavioral control.

1 INTRODUCTION

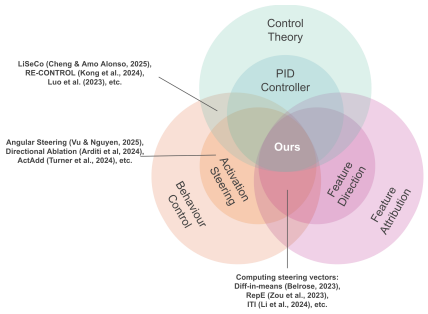

Figure 1: Our paper connects LLM Behavior Control, Feature Attribution for LLM and Control Theory. Specifically, we apply a PID-Controller to compute the steering vector for activation steering.

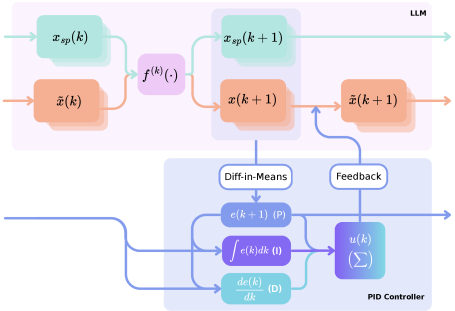

Figure 2: PID Steering: To compute the steering vector $u(k)$: a PID controller is applied at every layer $f^{(k)}(\cdot)$, using the diff-in-means between 2 contrastive data $x_{sp}(k)$ and $x(k)$ as the error signal $e(k)$.

Large language models (LLMs) have demonstrated remarkable capabilities across diverse domains, yet ensuring that their outputs align with desired behaviors remains a central challenge (Dang et al., 2025; Sclar et al., 2023; Kotha et al., 2023; Luo et al., 2025; Houlsby et al., 2019). Common post-training approaches (Wei et al., 2021; Ouyang et al., 2022) have proven effective for improving alignment. However, these techniques demand substantial computational resources (Houlsby et al., 2019)

* Co-first authors. † Co-last authors. Correspondence to: dungnv@u.nus.edu & tanmn@nus.edu.sg

and require weight updates with new training data, which can unintentionally degrade fluency or performance on unrelated tasks (Templeton et al., 2024; Kotha et al., 2023; Luo et al., 2025).

An increasingly popular alternative is *activation steering*, which modifies a model's internal activations directly at inference time, avoiding costly retraining (Vu & Nguyen, 2025; Li et al., 2024; Turner et al., 2023; 2024; Lee et al., 2024; Rimsky et al., 2024; Rodriguez et al., 2025). This approach has been employed both to probe internal representations (Geiger et al., 2024; von Rütte et al., 2024; Vu & Nguyen, 2025) and to enable fine-grained behavioral control (Vu & Nguyen, 2025; Rodriguez et al., 2025; Turner et al., 2024; Zou et al., 2023a; Rimsky et al., 2024; Li et al., 2024). Recent work demonstrates that steering along carefully chosen low-dimensional directions can effectively alter model behavior (Turner et al., 2024; Rimsky et al., 2024; Arditi et al., 2024; Zou et al., 2023a; Vu & Nguyen, 2025), highlighting its potential as a lightweight yet powerful alignment strategy.

Steering through the Lens of Dynamical Systems. Recent methods leverage the geometric structure of the activation space (Marks & Tegmark, 2024; Park et al., 2024) using linear algebraic techniques (Turner et al., 2024; Zou et al., 2023a; Rimsky et al., 2024; Arditi et al., 2024; Vu & Nguyen, 2025) to compute the steering vectors. While effective, these works oversimplify the complex, dynamic behavior arising from the auto-regressive nature of LLMs. When viewed through this dynamical lens, activation steering can be interpreted as guiding the model's trajectory through activation space, from a region encoding one concept to another, analogous to steering a dynamical system from one state to a desired target state.

Contribution. Building on the aforementioned dynamical system insight, our work departs from the prevailing algebraic framing and instead adopts a control-theoretic perspective on activation steering. Although recent studies (Soatto et al., 2023; Kong et al., 2024; Luo et al., 2023) have begun exploring this direction, their focus has primarily remained at the level of the token-level generation proceses, treating high-level behaviors as control signals. In contrast, we take into account the internal mechanisms of LLMs by modeling the layer-wise construction of feature directions (Bricken et al., 2023; Park et al., 2024) as a dynamical system. These feature directions are then used as steering vectors (Turner et al., 2024; Zou et al., 2023a; Rimsky et al., 2024; Arditi et al., 2024; Vu & Nguyen, 2025). Specifically, we show that existing steering methods relying on difference-of-means feature directions (Rimsky et al., 2024), including Activation Addition (ActAdd) (Turner et al., 2024), Directional Ablation (Arditi et al., 2024), and Mean Activation Transport (Mean-AcT) (Rodriguez et al., 2025), can be interpreted as instances of a *proportional (P) controller*, thus suffering from the steady-state error due to the disturbance to the state of the system (Åström & Hägglund, 1995b). This new perspective enables the application of principled control-theoretic strategies for extracting effective feature directions and computing steering vectors, thereby offering stronger robustness and performance guarantees for activation steering methods. An overview of our approach is shown in Fig. 1 and 2. In this paper, we use the terms feature direction and steering vector interchangeably, noting that steering vectors represent a practical application of feature directions in activation steering. Our contribution is three-fold:

1. **Control-Theoretic Formulation for Feature Direction:** We develop a new control-theoretic framework for constructing feature directions/steering vectors along the layers of an LLM.
2. **PID-Based Steering:** We propose the novel *Proportional-Integral-Derivative (PID) Steering*, a control-theoretic framework for computing feature directions using a PID controller to reduce the steady-state error inherent in existing activation steering methods (see Fig. 2).
3. **Unified Theoretical Framework:** We demonstrate that common activation steering methods correspond to proportional (P) controllers. This connection enables a theoretical analysis that highlights PID Steering's advantages in reducing steady-state error and oscillations

We comprehensively validate our PID Steering across diverse *modalities* (text and image), *downstream applications* (toxicity mitigation, jailbreaking attack, and image style control), *steering paradigms* (ActAdd, Mean-AcT, and Angular Steering (Vu & Nguyen, 2025)), *model families* (Qwen2.5 (Yang et al., 2024), Gemma2 (Gemma Team et al., 2024), Llama3 (Llama Team, 2024), SDXL-Lightning (Lin et al., 2024), and Flux (Labs, 2024)), and *model scales* (3B-14B for language models and 3.5B-12B for diffusion models).

Organization. We organize the paper as follows: Section 2 reviews background; Section 3 links activation steering to P control and introduces PID Steering; Section 4 presents its theoretical analysis; Section 5 provides empirical validation; Appendix A discusses related work; and Section 6 concludes. Proofs, derivations, and additional experiments are in the Appendix.

2 BACKGROUND

2.1 TRANSFORMERS

Decoder-only transformers take an input token sequence $\boldsymbol{q} = [q_1, \ldots, q_n]$ and map it to initial embeddings $\boldsymbol{x}(1) = [\boldsymbol{x}_1(1), \ldots, \boldsymbol{x}_n(1)]^\top = \text{Embed}(\boldsymbol{q})$. The embeddings are then propagated through K layers. At each layer k, the residual activation $\boldsymbol{x}_i(k)$ for token p_i is updated by self-attention and an MLP block, with normalization applied before (and sometimes after) these modules:

$$\boldsymbol{x}_{i,\text{post-attn}}(k) = \boldsymbol{x}_i(k) + \text{SelfAttn}^{(k)}(\text{Norm}(\boldsymbol{x}_i(k)))$$
$$\boldsymbol{x}_i(k+1) = \boldsymbol{x}_{i,\text{post-attn}}(k) + \text{MLP}^{(k)}(\text{Norm}(\boldsymbol{x}_{i,\text{post-attn}}(k))).$$

In this paper, for notational brevity, we summarize the layered processing above as $\boldsymbol{x}_i(k+1) = f_i^{(k)}(\boldsymbol{x}(k))$, $i = 1, \ldots, n$, where $f_i^{(k)}$ encapsulates both the Self-Attention mechanism and Multi-Layer Perceptron at layer k. Finally, the output activations from the last layer, $\boldsymbol{x}_i(L+1)$, are decoded over the model's vocabulary to get the next token $y_i = \text{Decode}(\boldsymbol{x}_i(L+1))$ for subsequent generation.

2.2 ACTIVATION STEERING

Features such as behaviors or concepts are hypothesized to align with (approximately) orthogonal directions in activation space (Park et al., 2024; Bereska & Gavves, 2024; Elhage et al., 2022). Activation steering leverages this by modifying hidden states at inference to amplify or suppress specific features (Bayat et al., 2025; Konen et al., 2024; Li et al., 2024; Marks et al., 2025; Templeton et al., 2024). Recent approaches operationalize this idea by constructing feature directions, which act as *steering vectors* $\boldsymbol{r}$ for adjusting hidden states. These steering vectors are computed as layerwise differences in mean activations between datasets with contrasting concepts (e.g., harmful vs. harmless), a *difference-in-means* approach (Rimsky et al., 2024), shown to effectively isolate salient feature directions (Turner et al., 2023; 2024; Arditi et al., 2024).

2.2.1 APPLYING THE STEERING VECTORS

Two popular activation steering approaches that use steering vectors are: *Activation Addition* (Turner et al., 2024), and *Directional Ablation* (Arditi et al., 2024). Both methods modify the token activation $\boldsymbol{x}(k)$ using the steering vector $\boldsymbol{r}(k)$ at layer k such that the activation expresses the target concept or behavior. By setting $\boldsymbol{x}(1, \boldsymbol{q}) = \text{Embed}(\boldsymbol{q})$ and $\boldsymbol{r}(1) = 0$, these methods apply the steering vectors $\boldsymbol{r}(k)$ to the activation $\boldsymbol{x}(k)$, $k = [K]$, at each layer via a steering function ρ_{steer} as follows:

$$\boldsymbol{x}(k-1, \boldsymbol{q}) = \rho_{\text{steer}}(\boldsymbol{x}(k-1, \boldsymbol{q}), \boldsymbol{r}(k-1)), \text{for } \boldsymbol{q} \in \mathcal{D}_{\text{source}} \tag{1}$$
$$\boldsymbol{x}(k, \boldsymbol{q}) = f^{(k)}(\boldsymbol{x}(k-1, \boldsymbol{q})), \text{for } \boldsymbol{q} \in \mathcal{D}_{\text{source}} \cup \mathcal{D}_{\text{target}}. \tag{2}$$

We discuss here the details on how to design the steering function ρ_{steer} for each method.

Activation Addition (ActAdd). ActAdd and sets $\rho_{\text{steer}}(\boldsymbol{x}(k), \boldsymbol{r}(k)) = \boldsymbol{x}(k) + \alpha \boldsymbol{r}(k)$, where the coefficient α controls the strength of the effect.

Directional Ablation (DirAblate). DirAblate removes the feature by projecting the token activation onto the orthogonal complement, $\rho_{\text{steer}}(\boldsymbol{x}(k), \boldsymbol{r}(k)) = \boldsymbol{x}(k) - \boldsymbol{r}(k)\,\boldsymbol{r}(k)^\top\,\boldsymbol{x}(k)$.

2.2.2 COMPUTING THE STEERING VECTORS

Non-sequential Mapping. Let us use the jailbreaking task as an example. In this task, we apply activation steering to force the LLM to respond to harmful prompts (Vu & Nguyen, 2025; Arditi et al., 2024). In order to compute the steering vectors, i.e., refusal direction, for each layer $k \in [K]$ and post-instruction token position $i \in I$, we calculate the mean activation $\boldsymbol{\mu}_{i,\text{target}}(k)$ for harmless prompts from $\mathcal{D}_{\text{target}}^{(\text{train})}$ and $\boldsymbol{\mu}_{i,\text{source}}(k)$ for harmful prompts from $\mathcal{D}_{\text{source}}^{(\text{train})}$:

$$\boldsymbol{\mu}_{i,\text{target}}(k) = \frac{1}{|\mathcal{D}_{\text{target}}^{(\text{train})}|} \sum_{\boldsymbol{q} \in \mathcal{D}_{\text{target}}^{(\text{train})}} \boldsymbol{x}_i(k, \boldsymbol{q}), \qquad \boldsymbol{\mu}_{i,\text{source}}(k) = \frac{1}{|\mathcal{D}_{\text{source}}^{(\text{train})}|} \sum_{\boldsymbol{q} \in \mathcal{D}_{\text{source}}^{(\text{train})}} \boldsymbol{x}_i(k, \boldsymbol{q}). \tag{3}$$

We then compute the difference-in-means vectors, $\boldsymbol{r}_i(k) = \boldsymbol{\mu}_{i,\text{target}}(k) - \boldsymbol{\mu}_{i,\text{source}}(k)$, and use them as steering vectors. Optionally, among the difference-in-means vector $\boldsymbol{r}_i(k)$ for each post-instruction token position $i \in I$ at layer k, we can select the single most effective vector $\boldsymbol{r}(k) = \text{Select}(\{\boldsymbol{r}_i(k)\}_{i \in I})$ from this set by evaluating each candidate vector over validation sets $\mathcal{D}_{\text{source}}^{(\text{val})}$ and $\mathcal{D}_{\text{target}}^{(\text{val})}$.

Sequential Mapping. A non-sequential mapping neglects the causal dependency across activations, where outputs from one layer are passed to the next, i.e., $\boldsymbol{x}_i(k+1) = f_i^{(k)}(\boldsymbol{x}(k))$. Consequently, any